Analyzing the Role of AI-Assisted Programming in Modern Software Development

Assessment 02

Student Name: W.G.S.E. Madushani

Student ID: s17498

Registration Number: 2023s20248

Selected System: Student Performance Management System

Module Name: IT3003(Advanced Programming Techniques)

Submission Date:15.07.2026

Table of Contents

1. Introduction

Artificial Intelligence is now becoming part of modern software development. Tools such as ChatGPT, GitHub Copilot, code completion assistants and debugging assistants can help developers write code, correct errors and improve the structure of a program. However, these tools should be used carefully because the final responsibility for the program still belongs to the developer.

For this assessment, I selected a Student Performance Management System using Java. I selected this topic because I am an undergraduate in Industrial Statistics and Mathematical Finance. Therefore, a system that stores marks, calculates averages and produces summary statistics is connected with my academic background. At the same time, the system is simple enough to implement using Java console programming, but it still includes useful programming concepts such as classes, objects, lists, methods, validation and file handling.

The same problem was developed using two approaches. Version A was developed as a traditional Java program with a simple structure. Version B was developed as an AI-assisted version, where AI suggestions were used to improve structure, error handling and readability. The purpose of this report is to explain both versions, compare the results and critically discuss the role of AI in programming education and industry.

Both versions were evaluated against the same functional requirements and test cases. The comparison considered compilation, correct calculations, invalid-input handling, code structure, readability, maintainability, and active development time. Version A and Version B were compiled using JDK 17

2. Task 1: Conceptual Understanding

2.1 What is AI-Assisted Programming?

AI-assisted programming means using artificial intelligence tools to support software development activities. These tools can suggest code, explain errors, generate functions, improve code structure, write comments, create test cases and help with debugging. In simple terms, AI-assisted programming is not the same as replacing the programmer. It is more like having a support tool that gives suggestions while the programmer still has to understand, check and test the final output.

In a normal programming process, a developer reads the problem, designs the solution, writes the code, tests the program and fixes errors. AI tools can help in many parts of this process. For example, if a developer is unsure how to validate user input in Java, an AI tool can suggest sample code. However, the developer must still decide whether that code is correct for the actual project.

2.2 Types of AI Coding Tools

Tool Type	Description	Example Use
Code generation tools	Generate code based on a natural language prompt.	Creating a Java method to calculate a weighted average.
Debugging assistants	Help identify errors and suggest possible fixes.	Explaining a NumberFormatException in Java.
Code completion tools	Suggest the next line or block of code while typing.	GitHub Copilot suggesting a loop or method body.
Refactoring tools	Improve code structure without changing the main output.	Separating one large Java class into service and repository classes.
Documentation tools	Generate comments, README files or explanations.	Creating documentation for project functions.
Testing support tools	Suggest test cases and edge cases.	Testing invalid marks below 0 or above 100.

2.3 Advantages in Software Development

- AI can reduce development time by generating a starting point for code.
- It can explain errors in simple language, which is helpful for students and beginners.
- It can suggest better software structure, such as separating classes according to responsibility. For example, in Version B of this project, the AI suggested separating the model, service, repository, and input responsibilities into dedicated classes.
- It can improve readability by suggesting clearer method names and comments.
- It can support learning because the student can ask why a piece of code works.
- It can help identify missing validation and possible edge cases.

2.4 Risks and Limitations

- AI can produce code that looks correct but contains logical errors.
- A student may become dependent on AI and reduce their own problem-solving ability.

- Generated code may include insecure practices if it is not reviewed properly.
- AI may not fully understand the exact marking requirements of an assignment.
- If the developer cannot explain the code, the learning outcome is reduced.
- AI-generated code may need modification to match the course language, coding style and lecturer expectations.

3. Task 2: Practical Implementation

3.1 Selected Programming Problem

The selected programming problem is a Student Performance Management System. This system allows an academic user to manage student marks and calculate basic performance statistics. The system is console-based and developed using Java, which matches the programming language used in the course module.

Main functions of the system:

- Add a student record with student ID, name, assignment marks and exam marks.
- View all student records in a formatted table.
- Search for a student using student ID.
- Update assignment and exam marks.
- Delete a student record.
- Calculate weighted average, grade, class average, highest average and lowest average.
- Save and load records using a CSV file.
- To evaluate academic performance, the system calculates a weighted average for each student using the following formula:
  Weighted Average = (Assignment Mark*0.40) + (Exam Mark*0.60)

3.2 Why this Problem is Suitable

This problem is suitable because it is not too simple and not too complex. It includes data input, validation, calculations, data storage and reporting. It also connects with my degree background because it uses marks, averages and basic summary statistics. Therefore, I was able to understand the business logic even though I do not have a pure computer science background.

- Handling Data (CRUD): The program allows you to do everything you need with student records- Create (add) them, Read (view) them, Update (change) marks, and Delete them.
- Smart Error Checking: It checks user inputs to make sure they are valid (for example, stopping the user from entering marks lower than 0 or higher than 100) so the program does not crash.
- Real Calculations: It calculates a student's final grade using a weighted formula:

  Weighted Average = (Assignment Mark*0.40) + (Exam Mark*0.60)
- Class Statistics: It scans all students to calculate class-wide data, including the highest mark, lowest mark, class average, and the overall pass rate.
- Saving and Loading (CSV): It saves all student data into a spreadsheet-like file (a CSV file) on your computer, so the data is not lost when you close the program.
- Quick Search: It allows the user to find any student instantly by typing their unique Student ID.
- Two Different Designs: It lets us compare a simple, beginner-friendly setup (Version A: everything in one file) with a professional, organized setup (Version B: code split into different files based on what they do).

3.3 Version A: Traditional Programming Approach

Version A was developed as a simple Java console application. The program is mainly written in one file named StudentPerformanceTraditional.java. This version uses basic Java features such as Scanner, ArrayList, methods, loops, conditional statements and file handling. The purpose of this version was to solve the problem in a direct way without focusing too much on advanced structure.

Main characteristics of Version A:

- Simple menu-driven structure.
- Student class is written as an inner class.
- All main logic is in one Java file.
- CSV file is used to save records.
- Input validation is included for empty text and marks between 0 and 100.

```
C:\Users\MSI\Downloads\AI_Assisted_Programming_Assessment_Java_Pack\AI_Assisted_Programming_Assessment_Java_Pack\Version_A_Traditional\src>javac StudentPerf
ormanceTraditional.java

C:\Users\MSI\Downloads\AI_Assisted_Programming_Assessment_Java_Pack\AI_Assisted_Programming_Assessment_Java_Pack\Version_A_Traditional\src>java StudentPerfo
rmanceTraditional

===== Student Performance Management System =====
1. Add Student
2. View All Students
3. Search Student by ID
4. Update Student Marks
5. Delete Student
6. Show Class Summary
7. Save and Exit
Enter your choice:
```

```
===== Student Performance Management System =====
1. Add Student
2. View All Students
3. Search Student by ID
4. Update Student Marks
5. Delete Student
6. Show Class Summary
7. Save and Exit
Enter your choice: 1
Student ID: 5
Student Name: Nimal
Assignment Marks (0-100): 23
Exam Marks (0-100): 56
Student added successfully.
```

```
===== Student Performance Management System =====
1. Add Student
2. View All Students
3. Search Student by ID
4. Update Student Marks
5. Delete Student
6. Show Class Summary
7. Save and Exit
Enter your choice: 2
ID          Name                    Assignment   Exam         Average    Grade
--------------------------------------------------------------------------
12          Sanduni                 34.00        45.00        40.60      D
1           Erandika                75.00        65.00        69.00      B
5           Nimal                   23.00        56.00        42.80      D
```

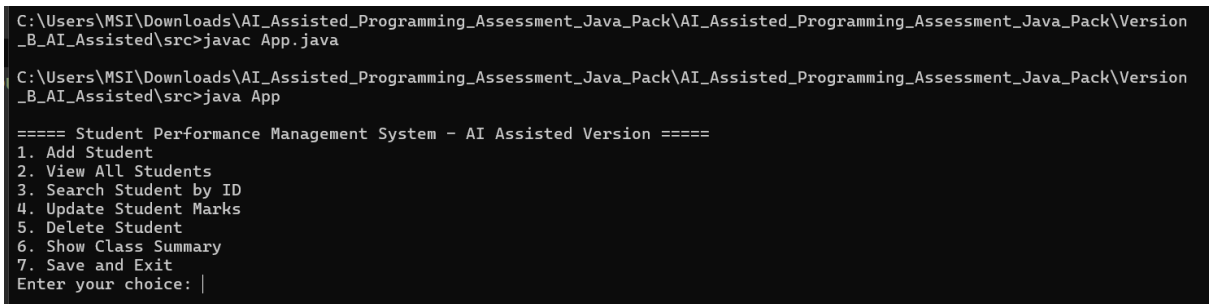
```
===== Class Summary =====
Number of students: 3
Class average: 50.80
Highest average: 69.00
Lowest average: 40.60
```

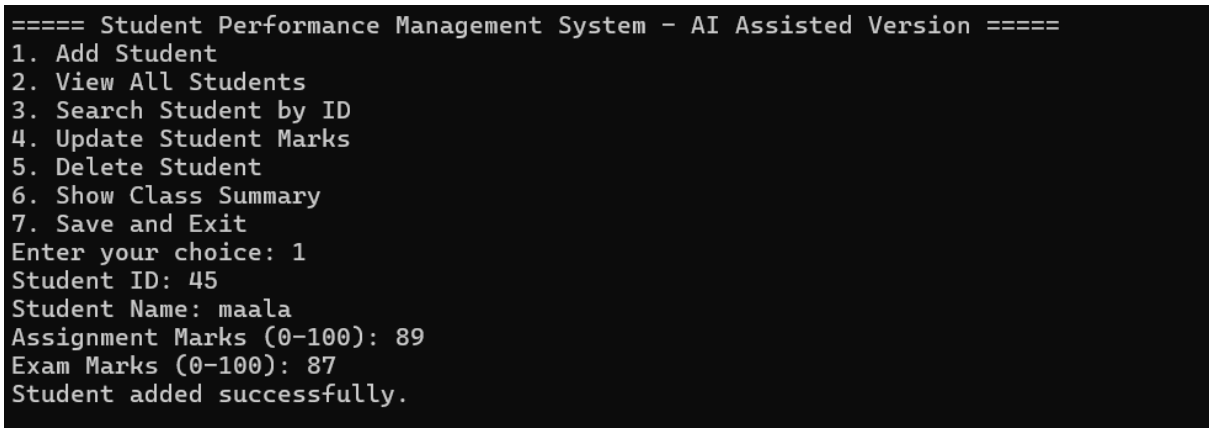
3.4 Version B: AI-Assisted Programming Approach

Version B was developed using the same problem but with AI assistance for improving the structure and maintainability of the code. The AI-assisted version separates responsibilities into different classes. This makes the code easier to read, update and test compared with keeping everything in one file.

Main classes in Version B:

Class Name	Responsibility	Representative Methods
App	Controls the menu and program flow.	main (String [] args)
StudentRecord	Stores student details and calculates average and grade.	calculateWeightedAverage(), calculateGrade()
StudentService	Handles business logic such as add, search, update, delete and summary.	addStudent(), findById(), updateMarks(), deleteStudent()
StudentRepository	Handles loading and saving data to CSV file.	load(), save()
StatisticsSummary	Stores calculated summary values.	calculateClassSummary()
InputHelper	Handles safe input reading and validation.	readRequiredText(), readMenuChoice(), readMark()





```
===== Student Performance Management System - AI Assisted Version =====
1. Add Student
2. View All Students
3. Search Student by ID
4. Update Student Marks
5. Delete Student
6. Show Class Summary
7. Save and Exit
Enter your choice: 2
ID          Name                    Assignment   Exam         Average    Grade
--------------------------------------------------------------------------
45          maala                   89.00        87.00        87.80      A
67          ravi                    76.00        65.00        69.40      B
```

```
===== Student Performance Management System - AI Assisted Version =====
1. Add Student
2. View All Students
3. Search Student by ID
4. Update Student Marks
5. Delete Student
6. Show Class Summary
7. Save and Exit
Enter your choice: 6

===== Class Summary =====
Number of students: 2
Class average: 78.60
Highest average: 87.80
Lowest average: 69.40
Pass rate: 100.00%
```

3.5 Code Sections: Manual and AI-Assisted

The following table explains which parts were manually written and which parts were improved with AI assistance. This is important because AI support should be clearly identified instead of hiding it.

Section	Manual Work	AI-Assisted Support
Problem selection	Selected student performance system based on my statistics background.	AI helped confirm that the topic is suitable for the assessment.
Version A structure	Manually planned a simple menu driven Java program.	The Java code in StudentPerformanceTraditional.java was written entirely manually without any AI-generated code.
Input validation	Added simple checks for empty fields and marks range.	AI suggested clearer validation and exception handling ideas.
Version B class design	Reviewed and selected class responsibilities.	AI suggested separating model, service, repository and input helper classes.
Summary calculations	Understood the calculation of average, highest, lowest and pass rate.	AI helped organize the calculation inside a separate StatisticsSummary class.
Documentation	Edited the explanation to match my project.	AI helped create a starting structure for README and report sections.

3.6 AI Prompts Used During Development

No.	AI Tool	Prompt Used	How the Output was Used
1	ChatGPT	Suggest a moderate Java console project suitable for an AI-assisted programming assessment.	Helped select the Student Performance Management System.
2	ChatGPT	Give me a simple Java class design for a student marks management system.	Helped improve the class structure for Version B.
3	ChatGPT	Improve input validation for marks between 0 and 100 in Java.	Helped refine validation logic.
4	ChatGPT	Explain how to compare manual and AI-assisted implementations.	Helped structure the comparison table.
5	ChatGPT	Check this Java code for readability and possible errors.	Helped identify small improvements before final testing.

3.7 Comparison of Traditional and AI-Assisted Approaches

Both implementations compiled successfully on JDK 17. Version A contains 271 source lines in one Java file, whereas Version B contains 431 source lines across six Java files. Version B therefore introduces more code and abstraction, but it separates user input, business logic, data storage, and summary reporting. Functional comparison should use the same test data and expected results for both versions. The comparison below summarizes the main differences.

Comparison Area	Version A: Traditional Java	Version B: AI-Assisted Java
Development time	Took more time because the structure, validation and calculations were written manually. [Approximately 4 hours].	Took less time to improve the structure because AI suggestions provided a starting point. [Approximately 2 hours].
Code quality	Functional and easy to follow, but most logic is in one file.	More organized because each class has a clear responsibility.
Error handling	Basic validation is included for menu choice and marks.	Validation and exception handling are more structured and easier to maintain.
Readability	Readable for a small program, but the file becomes longer as features increase.	More readable because related logic is separated into classes.
Maintainability	Changing one part may require editing the main file.	Easier to maintain because file handling, business logic and input are separated.
Learning value	Helped me understand the basic logic directly.	Helped me understand how the same logic can be written in a more professional structure.

3.8 UML Diagrams

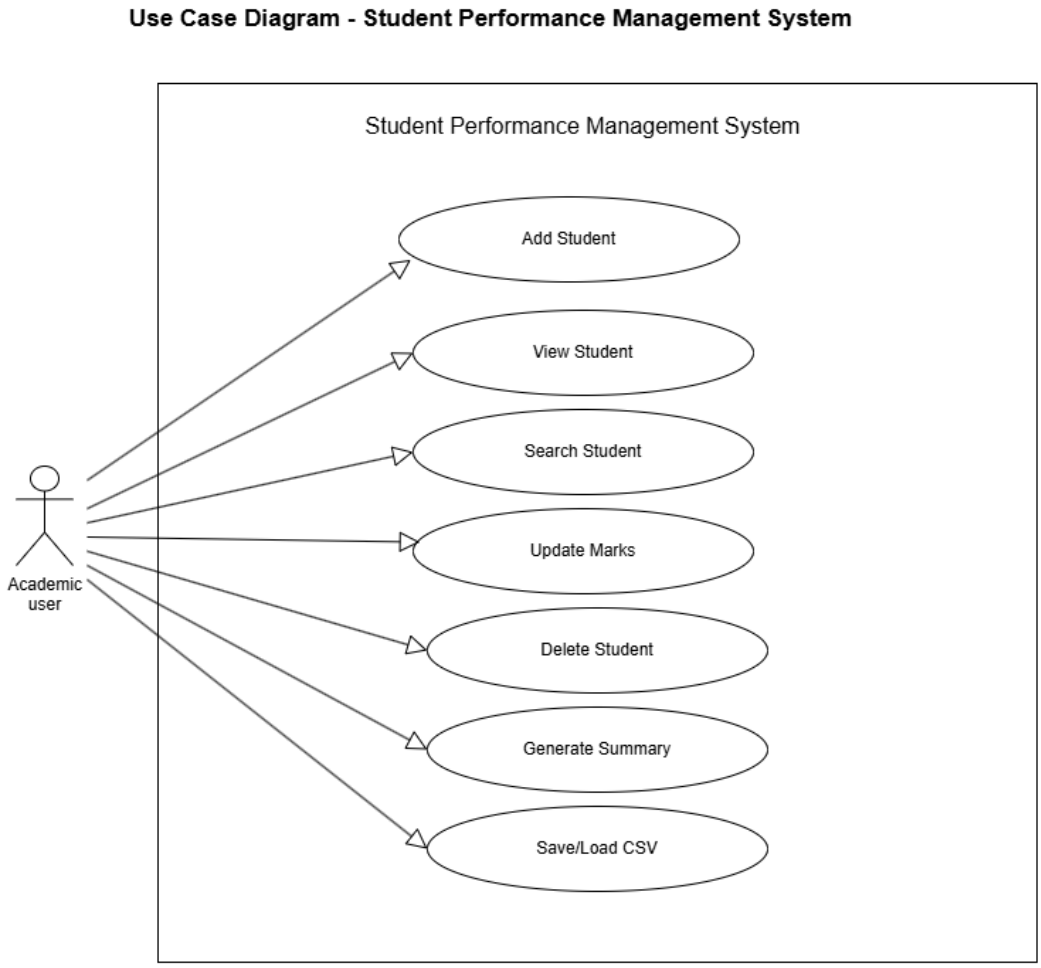


Figure 1: Use Case Diagram

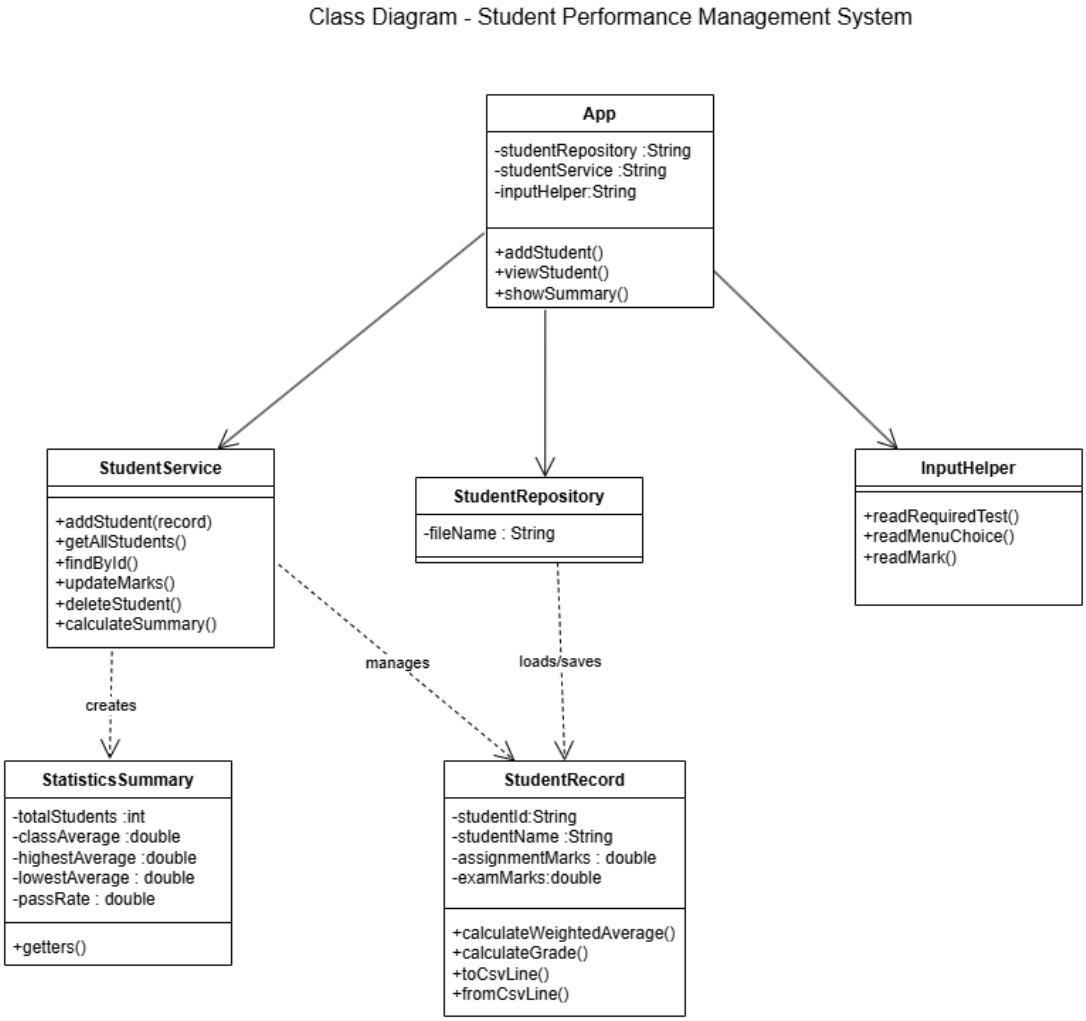


Figure 2: Class Diagram

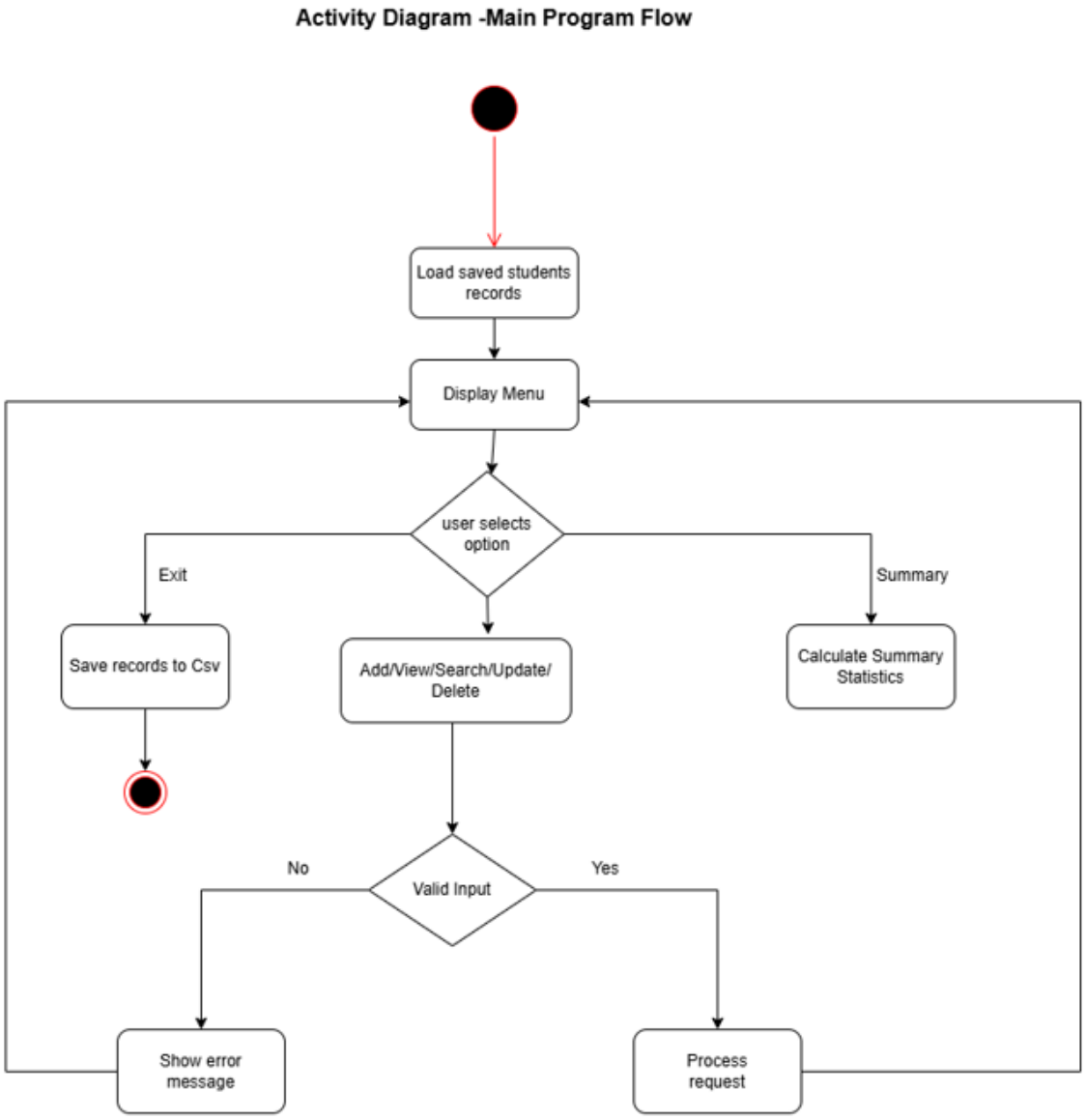


Figure 3: Activity Diagram

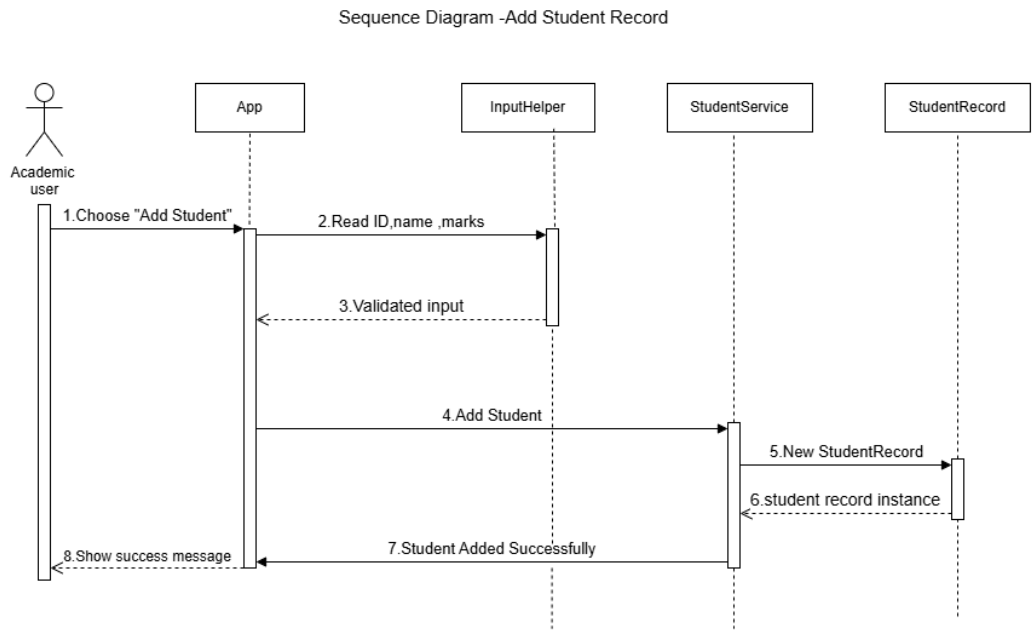


Figure 4: Sequence Diagram

4. Task 3: Critical Analysis

4.1 How AI Changed the Coding Approach

AI changed the coding approach mainly by helping me think about structure. In the traditional version, I focused on making the program work. In the AI-assisted version, I also considered how to organize the code better. For example, instead of keeping all logic in one file, the AI-assisted version separates the program into classes such as StudentRecord, StudentService and StudentRepository. This made the project look closer to a real software development approach.

4.2 Whether AI Improved or Reduced Understanding

In my opinion, AI can improve understanding if it is used as a learning support tool. For example, when I asked about validation or class structure, the explanation helped me understand why the code should be separated. However, AI can reduce understanding if a student copies the output without reading it. Therefore, I think the correct method is to ask questions, test the answer, modify the code and make sure every part can be explained.

4.3 Cases Where AI Should Not Be Used

- AI should not be used to submit work that the student does not understand.
- AI should not be used to generate insecure code for systems that handle private or financial data without expert review.
- AI should not be used when the assessment specifically says no AI assistance is allowed.
- AI should not be trusted blindly for legal, security or safety-critical software.
- AI should not replace proper testing and debugging by the developer.

4.4 Ethical Considerations in AI-Generated Code

There are several ethical points related to AI-generated code. First, the student should be transparent about which parts were supported by AI. Second, the final code should be tested carefully because AI can make mistakes. Third, AI tools should not be used to copy private or confidential information into an online system. Fourth, students should not use AI in a way that removes their own learning. In this project, AI was treated as a support tool for improving the Java structure and documentation, while the final review and understanding should remain with the student.

5. Task 4: Conclusion

AI-assisted programming is useful in both programming education and the software industry. It can save time, explain errors and suggest better ways to organize code. In this assessment, AI assistance helped improve the Java version by making it more modular and maintainable. However, the traditional version was also important because it helped build the basic understanding of the problem and logic.

My final opinion is that AI should be used as a learning partner, not as a replacement for the programmer. Students should use AI to ask questions, compare approaches and improve their code, but they must still understand and test the final program. In industry, AI can improve productivity, but human developers are still needed for design decisions, ethical judgement, security review and final responsibility.

6. References

- Ebert, C., & Louridas, P. (2023). Generative AI for software practitioners. IEEE Software, 40(4), 30–38. https://doi.org/10.1109/MS.2023.3265321

- Ziegler, A., Kalliamvakou, E., Li, X. A., Rice, A., Rifkin, D., Simister, S., ... & Aftandilian, E. (2022). Productivity assessment of neural code completion. In Proceedings of the 6th ACM SIGPLAN International Symposium on Machine Programming (pp. 21–29). Association for Computing Machinery. https://doi.org/10.1145/3532504.3534080

7. GitHub Repository Link

GitHub Repository Link: https://github.com/ERANDIKA0124/AI-Assisted-Programming